

Technical Learning Plan

Learning-oriented engineering topics for BankOne. These are **not all required for the current demo size**, but they build production / interview skills. Implement in phases; mark status as you go.

Status legend: Planned · In progress · Implemented (in BankOne) · Lab only

Last updated: 2026-07-31

Why this plan exists

BankOne already teaches domain banking (customers, accounts, ledger, portal, approvals, audit trail, Kafka email, reports). This document adds **platform engineering** skills: resilience, scale, security lifecycle, and observability.

Use BankOne as the sandbox — prefer small, demoable slices over big-bang rewrites.

Phase map (recommended order)

Phase A	Reliability at the edge Rate limiting → Idempotency → Circuit breaker / retry
Phase B	Data & messaging correctness Outbox → Kafka DLQ + consumer idempotency → Optimistic locking
Phase C	Performance & scale (without jumping to shards) Caching → Read replicas (concept) → Cursor pagination
Phase D	Product evolution API versioning → Feature flags → OpenAPI
Phase E	Observability & ops Correlation ID → Metrics → Tracing → Health → Flyway → CI
Phase F	Scale lab Shard router design → optional multi-DB lab

Core platform topics (original six + sharding)

1. Rate limiting — Planned

Use	Cap requests per client/IP/user so abuse or loops cannot overwhelm the API
BankOne hooks	Stricter limit on <code>POST /auth/login</code> ; general limit on <code>/portal/**</code> and money APIs
Learn	Token bucket (Bucket4j), HTTP 429 , <code>Retry-After</code> , in-memory vs Redis shared state
Demo	Burst login attempts → 429; UI still usable at normal speed

2. Caching — Planned

Use	Serve hot, rarely changing reads without hitting Postgres every time
BankOne hooks	Account policies, role → access catalog, static reference data
Learn	Cache-aside, TTL, eviction on policy/role update
Avoid caching	Live balances right after deposit (unless explicit invalidate)

3. Circuit breaker + retry — Planned

Use	When Kafka / email / future gateways are down, fail fast instead of blocking threads
BankOne hooks	<code>NotificationEventPublisher</code> , SendGrid/SMTP, Kafka producer
Learn	Resilience4j open / half-open / closed; timeout budgets; retry with jitter
Note	BankOne already soft-fails Kafka locally via <code>APP_KAFKA_ENABLED</code> / short block ms — circuit breaker formalizes this

4. Idempotency keys — Planned (high banking value)

Use	Double-click / network retry must not double-post money
BankOne hooks	Deposit, withdraw, transfer, portal transfer, approve transfer
Learn	Client <code>Idempotency-Key</code> header; server store key → response; TTL
Demo	Same key twice → same result, one ledger row

5. Transactional outbox — Planned

Use	Persist “notify later” in the same DB transaction as the money change
BankOne hooks	Account open / deposit / transfer → notification
Learn	Outbox table + poller/CDC → Kafka; at-least-once with consumer idempotency
Why	Avoid “balance updated but email never queued” races

6. API versioning + feature flags — Planned

Use	Evolve contracts and roll out features without breaking Angular
BankOne hooks	<code>/v1/...</code> paths; flags for PDF reports, new approval rules
Learn	URL vs header versioning; flag stores; gradual rollout

7. Sharding — Lab only (later)

Use	Split data across DBs when one Postgres cannot hold/serve all customers
BankOne approach	Design <code>ShardRouter</code> by <code>customerId % N</code> first; still one DB until needed
Hard part	Cross-shard transfers (distributed consistency) — do not start here
Earlier alternative	Read replicas + caching + indexes usually come before shards

Extended learning topics (include all)

Security & identity

Topic	Use	BankOne hook	Status
JWT refresh + rotation	Short-lived access; revoke after lock/password change	Auth + deferred hardening	Planned
Secrets management	No secrets in git / properties for prod	JWT_SECRET , DB password env	Partial (env placeholders)
Validation & OWASP basics	Reject bad input early	@Valid on create/login/open	Partial
CORS / cookie CSRF awareness	Browser security model	Already CORS; stay JWT Bearer	Implemented (CORS)

Reliability & data

Topic	Use	BankOne hook	Status
Optimistic locking (@Version)	Concurrent edits without silent overwrite	AuditableEntity.version ; accounts	Partial (entity field)
Pessimistic locks	Serialize money moves	findByIdForUpdate on withdraw/transfer	Implemented
Saga / compensation	Multi-step other-bank settlement	Approve OTHER_BANK ≠ ledger yet	Planned
Flyway / Liquibase	Versioned schema instead of only ddl-auto	New tables (audit_event , ...)	Planned
Read replicas (concept)	Scale reporting reads	Reports / audit list	Planned

Observability

Topic	Use	BankOne hook	Status
Correlation ID	Trace one request across logs	Filter → MDC → Kafka headers	Planned
Metrics (Micrometer)	Count 429s, transfer latency, login fails	Actuator already on classpath	Partial (Actuator dep)
OpenTelemetry tracing	Follow API → DB → Kafka → mail	Notification path	Planned
Health checks	Probe DB/Kafka for ops	Actuator health	Partial

Messaging (Kafka path)

Topic	Use	BankOne hook	Status
Dead-letter queue	Poison messages stop blocking consumers	Notification consumer	Planned
Consumer idempotency	At-least-once must not spam duplicate emails	Notification listener	Planned
Scheduled jobs	Interest, purge old audit, outbox poller	@Scheduled	Planned

API / frontend contract

Topic	Use	BankOne hook	Status
OpenAPI (springdoc)	Live API docs + client gen	Controllers	Planned
Cursor pagination	Stable paging on huge tables	Transactions / audit events	Planned
ETags / conditional GET	Cheap revalidation	Customer/account detail	Planned

Architecture patterns

Topic	Use	BankOne hook	Status
Light CQRS	Separate write path vs report aggregates	com.bankone.report	Partial (reports read path)
Ports & adapters	Test domain without HTTP/JPA	Transfer / account services	Planned
Multi-tenancy concept	Branch/bank isolation	branchCode → tenant	Planned

Ops & deploy

Topic	Use	BankOne hook	Status
Full Docker Compose	App + Postgres + Redis + Kafka + Mailpit	Local prod-like	Partial (Kafka/Mailpit)
CI pipeline	Build/test WAR on every push	GitHub Actions	Planned
Blue/green idea	Zero-downtime mental model	Liberty / Render	Planned

Already implemented in BankOne (related foundations)

Use these as **starting points** when implementing the plan:

Foundation	Where
JWT + role/access security	<code>auth</code> , <code>SecurityConfig</code> , <code>AppAccess</code>
Money locks + ledger	<code>AccountServiceImpl</code> , <code>Transaction</code>
Kafka → email notifications	<code>com.bankone.notification</code>
Activity audit + backfill	<code>com.bankone.audit</code>
Portal + beneficiaries + approvals	<code>portal</code> , <code>beneficiary</code> , <code>transfer</code>
Reports + PDF	<code>com.bankone.report</code>
Actuator dependency	<code>pom.xml</code>

Suggested build sequence (concrete)

1. **Correlation ID filter** (small, high clarity)
2. **Rate limit login** (Bucket4j in-memory)
3. **Idempotency on deposit/transfer**
4. **Circuit breaker around Kafka/mail publish**
5. **Redis cache for account policies**
6. **Outbox table + poller** (replace fire-and-forget publish)
7. **Flyway** for next schema change
8. **DLQ + consumer idempotency**
9. **OpenAPI**
10. **Sharding lab** (router interface only, then optional 2nd DB)

How to use this doc while coding

1. Pick **one** topic per session.
2. Add a thin vertical slice (filter/service + config + curl demo).
3. Update this file's Status column + [CHANGELOG.md](#).
4. Link the demo steps under DEVELOPMENT_GUIDE when the feature lands.
5. Do **not** start deferred hardening (`.cursor/rules/deferred-hardening.mdc`) unless explicitly requested — that list is production correctness, separate from this learning track.

Interview one-liners

- **Rate limit:** protect capacity and login from abuse.
- **Idempotency:** retries safe for money.
- **Outbox:** DB and messaging stay consistent.
- **Circuit breaker:** unhealthy dependency must not freeze the bank API.
- **Cache:** speed reads you can afford to be slightly stale or invalidate.
- **Shard:** last resort after vertical scale, indexes, replicas, and caching.